

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

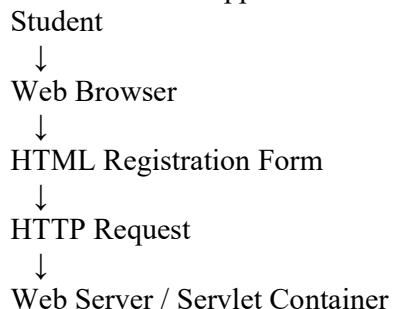
Answer:

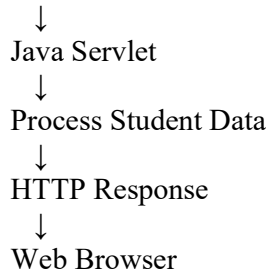
Scenario

A college wants an online student registration system where students enter their details through a web form and the data is processed using a Java Servlet.

1. Servlet Architecture

A Servlet-based application follows this basic architecture:





The **Servlet Container** receives the HTTP request and forwards it to the appropriate Servlet. The Servlet processes the request and generates a response.

2. Servlet Using GET and POST

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentServlet extends HttpServlet {

    // Handles GET request
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String regno = request.getParameter("regno");

        out.println("<h2>Student Details</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Register Number: " + regno);
    }

    // Handles POST request
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String regno = request.getParameter("regno");
        String email = request.getParameter("email");

        out.println("<h2>Registration Successful</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Register Number: " + regno + "<br>");
        out.println("Email: " + email);
    }
}
```

```
}
```

3. Servlet Life Cycle

The Servlet has three important life-cycle methods:

init()

- Called only once when the Servlet is created.
- Used for initialization tasks.

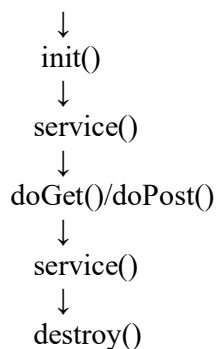
service()

- Called whenever a request is received.
- Processes the request and generates a response.
- It internally calls methods such as doGet() or doPost().

destroy()

- Called once before the Servlet is removed.
- Used to release resources.

Servlet Created



4. GET vs POST

GET

Data is sent through URL
Data is visible in URL
Suitable for retrieving data

POST

Data is sent in request body
Data is not displayed in URL
Suitable for submitting data

Less suitable for sensitive information Better choice for sensitive data

POST is more appropriate for sensitive data because the submitted information is not exposed directly in the URL. However, POST alone does not provide encryption; **HTTPS should be used** to protect sensitive information.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies. The scenario requires users to log in and remain authenticated while moving between multiple pages. The assessment specifically asks for **HttpSession and Cookies** and a comparison of session tracking methods.

Answer:

1. Login Servlet Using HttpSession

```
import java.io.*;
import javax.servlet.*;
```

```

import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (username.equals("admin") && password.equals("1234")) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);

            response.sendRedirect("home.jsp");

        } else {
            response.getWriter().println("Invalid username or password");
        }
    }
}

```

2. Maintaining the Session

After successful login, the username can be retrieved from the session on another page:

```
HttpSession session = request.getSession(false);
```

```

if (session != null) {
    String username =
        (String) session.getAttribute("username");

    response.getWriter().println(
        "Welcome " + username
    );
}

```

The session allows the server to remember the user between different HTTP requests.

3. Using Cookies

A cookie can store information on the user's browser.

```
Cookie cookie = new Cookie("username", username);
```

```
cookie.setMaxAge(60 * 60);
```

```
response.addCookie(cookie);
```

To retrieve the cookie:

```
Cookie[] cookies = request.getCookies();
```

```

if (cookies != null) {
    for (Cookie cookie : cookies) {

        if (cookie.getName().equals("username")) {
            String username = cookie.getValue();

```

```

        response.getWriter().println(
            "Welcome " + username
        );
    }
}
}

```

4. How Session Tracking Works

HTTP is **stateless**, meaning the server does not automatically remember previous requests.

Session tracking solves this problem:

User Login



Server Creates Session



Session ID Generated



Browser Stores Session ID



User Visits Another Page



Session ID Sent Again



Server Finds User's Session

5. HttpSession vs Cookies

HttpSession

Session data is maintained on server

More suitable for login sessions

Session ID is associated with the user

Server controls session data

Can be invalidated using session.invalidate()

Cookies

Data is stored on client/browser

Suitable for preferences and simple information

Cookie contains name-value information

User's browser stores cookie

Can expire based on cookie lifetime

Conclusion

For a login system, **HttpSession is generally preferred for maintaining authenticated user information**, while cookies can assist with client-side persistence and session identification.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Answer:

The scenario requires an e-commerce application that can **connect to a database, insert product information, and retrieve product information using SQL queries**. It also requires JDBC architecture, connectivity steps, and exception handling.

1. JDBC Architecture

Java Web Application



JDBC API



JDBC Driver Manager



JDBC Driver



Database

JDBC (**Java Database Connectivity**) allows Java applications to communicate with relational databases.

2. Database Table

Example MySQL table:

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE products (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    category VARCHAR(50),  
    price DOUBLE,  
    quantity INT  
);
```

3. JDBC Connection

```
import java.sql.*;
```

```
public class DatabaseConnection {  
  
    public static Connection getConnection()  
        throws SQLException {  
  
        String url =  
            "jdbc:mysql://localhost:3306/ecommerce";  
  
        String username = "root";  
        String password = "root";  
  
        return DriverManager.getConnection(  
            url, username, password  
        );  
    }  
}
```

4. Insert Product

```
import java.sql.*;
```

```

public class ProductInsert {

    public static void main(String[] args) {

        String sql =
            "INSERT INTO products(name, category, price, quantity) "
            + "VALUES (?, ?, ?, ?)";

        try (Connection con =
            DatabaseConnection.getConnection();

            PreparedStatement ps =
                con.prepareStatement(sql)) {

            ps.setString(1, "Laptop");
            ps.setString(2, "Electronics");
            ps.setDouble(3, 55000);
            ps.setInt(4, 5);

            int rows = ps.executeUpdate();

            System.out.println(
                rows + " product inserted successfully."
            );

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

PreparedStatement is used because it allows parameters to be safely supplied to SQL statements and helps prevent SQL injection.

5. Retrieve Products

```

import java.sql.*;

public class ProductRetrieve {

    public static void main(String[] args) {

        String sql = "SELECT * FROM products";

        try (Connection con =
            DatabaseConnection.getConnection();

            Statement stmt =
                con.createStatement();

            ResultSet rs =
                stmt.executeQuery(sql)) {

```

```

while (rs.next()) {

    int id = rs.getInt("id");
    String name = rs.getString("name");
    String category = rs.getString("category");
    double price = rs.getDouble("price");
    int quantity = rs.getInt("quantity");

    System.out.println(
        id + " | " + name + " | "
        + category + " | ₹" + price
        + " | Qty: " + quantity
    );
}

} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

6. JDBC Connectivity Steps

The standard steps are:

1. Load/Register JDBC Driver
- ↓
2. Establish Database Connection
- ↓
3. Create Statement / PreparedStatement
- ↓
4. Execute SQL Query
- ↓
5. Process ResultSet
- ↓
6. Handle Exceptions
- ↓
7. Close Database Resources

Exception Handling

JDBC operations can produce SQLException, so database code should use try-catch or try-with-resources.

```

try {
    // JDBC operations
}
catch (SQLException e) {
    System.out.println(
        "Database error: " + e.getMessage()
    );
}

```

Using **try-with-resources** is preferable because connections, statements, and result sets are automatically closed.

Code of Conduct:

I Sairaj C (192411018) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sairaj C

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand